

Génie Informatique

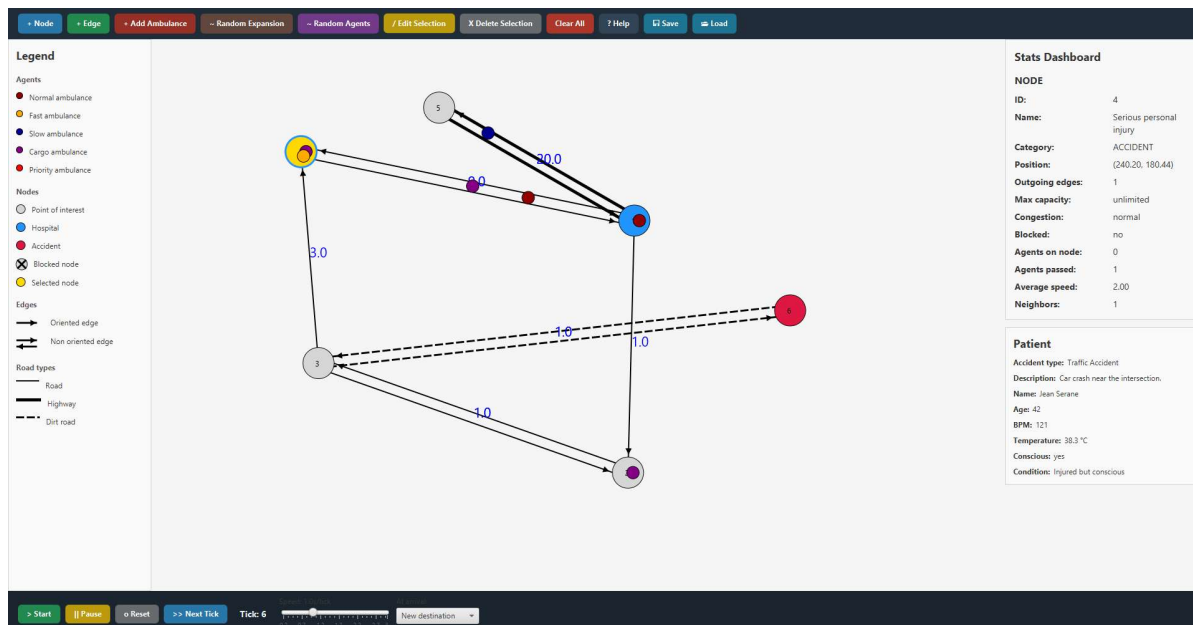
Encadré par Mme. HAWARI Sirine

GI-5 Groupe A



Rapport de projet

LifeLine GPS – Simulation d'agents dans un graphe



Réalisé par :

Romain MICHAUT-JOYEUX

Bilal MESBAHI

Nasser MOHAMED-ELMI

Jean-Luc MASLANKLA

Mohamed-Amine SBAIBY

Année universitaire 2025–2026

Sommaire

Introduction.....	2
I. Organisation du projet.....	3
II. Phase de développement.....	8
III. Fonctionnement du projet.....	13
IV. Difficultés rencontrées.....	21
V. Pistes d'améliorations.....	23
VI. Résultat final.....	24
Conclusion.....	26
Annexe (diagrammes).....	27

LifeLine GPS est une application de simulation développée dans le cadre de notre projet de génie logiciel à CY Tech. Elle modélise le déplacement d'agents autonomes (véhicules d'ambulance avec différentes caractéristiques) au sein d'un réseau routier cartographié sous forme de graphe pondéré.

L'objectif principal est de simuler de manière réaliste la circulation dans un réseau en tenant compte de contraintes variées : types de routes, capacités, congestion, et comportements propres à chaque agent. L'application offre une **interface JavaFX interactive** permettant à l'utilisateur de construire et modifier le graphe en temps réel, de piloter la simulation, et d'observer les comportements du trafic.

Ce projet nous a permis de mettre en pratique les principes du génie logiciel : programmation orientée objet, travail collaboratif avec **Git**, séparation des responsabilités (**jira**), et développement itératif guidé par un cahier des charges.

I. Organisation du projet

A. Outils et méthodes de travail

Pour mener à bien le développement de notre projet et atteindre les objectifs du cahier des charges, notre équipe a mis en place une structure organisationnelle et un flux de travail collaboratif bien défini. Nous avons optimisé notre gestion interne en nous appuyant sur des outils numériques adaptés :

- **Communication interne (Discord)** : nous avons configuré un serveur Discord comme environnement principal de travail. Cet espace nous a permis de centraliser nos discussions techniques, de partager nos progrès et de collaborer efficacement au quotidien avec un salon textuel pour prévoir les réunions, et un salon pour envoyer nos codes.
- **Gestion des tâches (Jira)** : afin de structurer précisément notre flux de travail, nous avons utilisé un projet Jira. Cet outil de gestion nous a permis de découper le projet en sous-tâches, de planifier les différentes fonctionnalités à implémenter et de suivre en temps réel l'avancement de chacun pour respecter les jalons imposés.
- **Gestion des versions (Git & GitHub)** : l'intégralité du code source a été centralisée sur un dépôt GitHub. Pour éviter toute perte de travail et fluidifier l'intégration de nos différents modules, nous avons adopté une gestion rigoureuse par branches. Cette approche nous a permis de développer et de tester de nouvelles fonctionnalités de manière isolée sans perturber le reste du projet, garantissant ainsi un environnement de travail constamment stable avant chaque fusion.
- **Suivi avec la tutrice** : dans le cadre de l'encadrement du projet, nous avons organisé une réunion par semaine avec notre tutrice, sur Teams, afin de rendre compte de l'avancement du projet et de voir les points à améliorer. Ces échanges nous ont permis d'obtenir un point de vue technique extérieur concernant notre application, ce qui nous a poussé à la rendre la plus fonctionnelle et accessible possible.

B. Répartition des tâches

Voici un tableau récapitulatif des tâches que chaque membre de l'équipe a réalisé :

Membre	Tâches
Bilal	• Implémentation des algorithmes (Dijkstra, A* et CongestionAware) • Implémentation des règles de déplacement selon les types d'agents et d'arêtes • Ajout slider pour rendre plus ou moins fluide la simulation • Utilisation du formulaire créé par Romain pour la création des objets • Corrections de bugs : Ralentissement de vitesse au cours du chemin • Refactorisation final du projet : restructuration des packages
Jean-Luc	• Modélisation diagrammes UML et Use Case • Intro, parties I et III du rapport • Fonctions pour enregistrer et lire la simulation dans un fichier
Nasser	• Création des classes de base (Agent, Edge, Graph, GraphController, SimulationController) • Système de position, mouvement et destination des agents (MVC) • Réorganisation des branches Git • Gestion des conflits entre agents et sur les arêtes de capacité

	• Correction de bugs (suppression d'agents, modes de création, agents prioritaires)
Mohamed-Amine	• Création des types d'agents (NormalAgent, SlowAgent, CargoAgent, PriorityAgent, AgentFactory) • Amélioration visuelle de l'interface (thème sombre, couleurs, effets hover) • Redesign de la barre d'outils (ToolBox) et de la barre de simulation (SimulationBar) • Refonte du rendu graphique (nœuds, arêtes, agents) • Refonte de la fenêtre d'aide (HelpDialog)
Romain	• Gestion des interactions utilisateurs (ajout d'un nœud, sélection, drag, clearAll, etc.) • Types de nœuds • Synchronisation algo et interface • Gestion des boutons JavaFX • Aspect visuel et formes (différents types de flèches, couleurs, etc.) • Panneaux légende, statistiques, patient, manuel utilisateur • Accidents et patients (lien avec Design) • Mise en forme et rédaction du rapport • Commentaires javadoc

C. Choix d'architecture

- L'architecture de *LifeLine GPS* repose sur une architecture MVC (Model View Controller). Le modèle regroupe les entités métier du projet : le graphe (Graph, Node, Edge) et le système d'agents (Agent et ses déclinaisons). La vue est structurée en composants JavaFX indépendants (GraphView, StatsPanel, PatientPanel, ToolBox...). Le contrôleur (GraphController, SimulationController) orchestre les interactions entre ces deux couches en maintenant la cohérence entre l'état du modèle et son affichage.
- Le package pathfinding a été séparé du modèle afin d'isoler la partie algorithmique. Cette décision permet d'ajouter ou modifier un algorithme sans impacter la logique métier, matérialisée par l'interface IPathFinder et la fabrique PathFinderFactory.
- Enfin, la simulation repose sur un modèle temporel discret tick-based : à chaque tick, le moteur traite séquentiellement les agents selon un ordre de priorité strict. Ce choix garantit un contrôle précis des états, la reproductibilité des scénarios et l'exécution pas-à-pas.

D. Limites du système

- Bien que fonctionnel, le système présente quelques limites : les données médicales des patients (BPM, condition, etc.) sont affichées dans l'interface mais n'influencent pas le calcul des chemins ni la priorisation des agents, ce qui constitue la limite fonctionnelle principale du projet.
- L'heuristique utilisée par A* est basée sur la distance euclidienne entre coordonnées écran et non sur des distances réelles, ce qui peut conduire à des estimations imprécises selon la disposition du graphe.
- Le système ne gère pas les graphes non connexes : si un nœud est isolé ou qu'un blocage coupe le graphe en deux, l'agent concerné reste bloqué sans qu'aucun message explicite n'en informe l'utilisateur.

- Enfin, aucune limite de performance n'a été testée sur de grands graphes. Le recalcul de chemin à chaque tick et le rendu JavaFX pourraient engendrer des ralentissements sensibles au-delà d'un certain nombre de nœuds et d'agents simultanés.

II. Phases de développement

A. MVP

Les objectifs du MVP était de poser les bases de l'application : modélisation d'un graphe, évolution d'un agent sur le graphe et affichage sur une interface JavaFX minimale.

Les fonctionnalités qui ont été implémentées sont les suivantes :

- Structure du graphe avec **Node**, **Edge** et **Graph** (liste d'adjacence)
- Premier agent avec déplacement basique via **Dijkstra**
- Moteur de simulation tick-based (SimulationEngine) avec méthode **tick()**
- Interface JavaFX minimale affichant le graphe et les agents
- Contrôles de base : **Start**, **Pause**, **Reset**, **Next Tick**

Nous pouvions voir un agent se déplacer de nœud en nœud sur un graphe simple tick par tick avec un affichage visuel fonctionnel.

B. Moteur de simulation

Nous devons rendre le moteur de simulation plus réaliste en gérant correctement la vitesse, la capacité des arêtes et les algorithmes de pathfinding.

Les fonctionnalités implémentées et corrigés sont les suivantes :

- Correction de perte de vitesse aux nœuds intermédiaires (boucle while remainingSpeed > 0)
- Ajout de **A*** comme second algorithme de pathfinding
- Choix de l'algorithme par agent (chaque agent possède son propre **lpathFinder**)
- Capacité des arêtes avec état WAITING quand une arête est saturée.

Ainsi les agents pouvaient se déplacer de façon cohérente en traversant plusieurs arêtes en un tick si leur vitesse le permettait.

C. Types et propriétés

Nous devons affiner le comportement des agents et des éléments du graphe en gérant correctement leurs propriétés et les interactions entre eux.

Les fonctionnalités implémentées et corrigées sont les suivantes :

- Gestion des conflits lorsque plusieurs agents tentent d'occuper simultanément le même nœud
- Résolution des conflits sur les arêtes dont la capacité est atteinte
- Correction du mécanisme de suppression d'agents pour éviter les incohérences d'état
- Désactivation automatique des modes de création lors de la sélection d'un élément existant du graphe
- Mise en place d'un système de priorité entre agents en situation de compétition

D. Interaction et édition

L'objectif de cette troisième phase d'amélioration était d'enrichir l'interface graphique JavaFX pour offrir une manipulation dynamique, intuitive et complète du graphe et de la simulation par l'utilisateur en temps réel.

Les fonctionnalités implémentées et corrigées sont les suivantes :

- Ajout de la possibilité pour l'utilisateur de construire son réseau routier à la volée en ajoutant de nouveaux nœuds (**+ Node**) ou en traçant des arêtes (**+ Edge**) directement depuis la barre d'outils supérieure de l'application.
- Mise en place d'un système permettant de cliquer sur un nœud ou une arête existante pour afficher ses propriétés détaillées dans le tableau de bord latéral (**Stats Dashboard**). L'activation du mode d'édition (**/ Edit Selection**) permet de modifier ces paramètres (comme le poids, la capacité ou le type d'une route) ou de supprimer l'élément (**X Delete Selection**).

- Implémentation de la gestion des interactions à la souris permettant de cliquer sur un nœud et de le glisser-déposer afin de réorganiser visuellement la topologie du graphe sur la zone de dessin principale.
- Intégration de boutons permettant d'ajouter manuellement une ambulance spécifique (**Add Ambulance**) ou de générer massivement des véhicules de manière aléatoire (**Random Agents**) pour observer instantanément l'impact du trafic sur le réseau.
- Ajout des fonctionnalités de persistance (**Save** et **Load**) permettant d'exporter la configuration complète d'un graphe et de sa simulation actuelle dans un fichier de données, afin de pouvoir la recharger ultérieurement.

E. Interface et visualisation

L'objectif était de finaliser l'interface utilisateur, améliorer la lisibilité visuelle et consolider la base de code.

Fonctionnalités implémentées :

- **StatsPanel** affichant les statistiques de l'élément sélectionné (nœud, arête, agent)
- **ToolBox** stylisée avec couleurs et effets de survol
- Sauvegarde et chargement de scénarios (**SaveManager**)
- Restructuration complète des packages pour une meilleure lisibilité du code

L'application atteint un niveau de finition suffisant pour être présentée et testée par un tiers. Le code est mieux organisé et documenté, facilitant les évolutions futures.

F. Diagrammes UML et Use case

Avant de coder, nous avons des diagrammes structurels et fonctionnels très simples. Néanmoins, nous avons réalisé une modélisation plus complète *a posteriori* qui nous a permis de consolider la structure de notre code et d'en valider la cohérence à travers deux documents majeurs :

1. Diagramme de cas d'utilisation (Use Case Diagram)

Ce diagramme a permis de définir les frontières du système et les interactions entre les utilisateurs et l'application. Il formalise les fonctionnalités opérationnelles essentielles :

- L'édition et la configuration du réseau routier (ajout/suppression de nœuds et d'arêtes).
- Le pilotage et le contrôle du moteur de simulation (lancement, pause, réinitialisation et exécution au pas-à-pas).
- La configuration et la génération des flux d'agents et d'accidents.

En identifiant clairement les besoins fonctionnels dès le départ, ce document a servi de fil conducteur pour le découpage de nos tâches sur Jira et la validation progressive de nos différents MVP.

2. Diagramme de classes (Class Diagram)

Ce document spécifie l'architecture orientée objet de l'application et assure le respect des principes de conception du génie logiciel. Il structure le projet en trois grands packages distincts :

- **Le modèle de l'environnement (Graph)** : représenté par la classe centrale Graph qui encapsule les entités Node (gérant les types HOSPITAL, ACCIDENT, POINT_OF_INTEREST) et Edge (définissant les routes, autoroutes, capacités et distances).
- **Le système d'agents autonomes (Agent)** : structuré autour d'une classe abstraite Agent et décliné en spécialisations concrètes selon les besoins du trafic (NormalAgent, FastAgent, SlowAgent, CargoAgent, PriorityAgent). Ce modèle intègre un cycle d'états strict (MOVING, WAITING, ARRIVED) pour réguler le comportement en fonction de la congestion.
- **Le moteur de simulation et le Pathfinding** : orchestré par la classe SimulationEngine (méthode tick() et moveAgent()). L'utilisation de l'interface IPathFinder et de la fabrique PathFinderFactory découple proprement les algorithmes de recherche de chemin (avec les algo DijkstraPathFinder, AStarPathFinder, et CongestionAwarePathFinder) de la logique interne des agents, permettant une extensibilité maximale du code.

Vous pourrez retrouver ces diagrammes en annexe de ce rapport.

III. Fonctionnement du projet

Ce projet consiste en une application JavaFX permettant de modéliser, visualiser et piloter le déplacement d'agents autonomes au sein d'un environnement cartographié sous forme de graphe pondéré. Le cœur du système repose sur une synchronisation temporelle couplée à des algorithmes de recherche de chemin.

A. Modélisation de l'environnement

L'environnement dans lequel évoluent les agents est représenté par la classe Graph. D'un point de vue structurel :

- **Structure de données** : le graphe utilise une structure de liste d'adjacence implémentée via une table de hachage. Cette approche optimise la recherche des voisins et la manipulation des connexions.
- **Noeuds (Node)** : ils représentent les intersections ou les positions physiques fixes où les agents peuvent stationner.
- **Arêtes (Edge)** : elles lient les nœuds entre eux. Le modèle gère à la fois des arêtes orientées et non orientées (dans ce dernier cas, une arête inverse est automatiquement générée lors de l'ajout). Chaque arête possède une distance (poids), une capacité maximale d'agents simultanés, ainsi qu'un type de route spécifique (autoroute, route classique, chemin de terre).

B. Moteur de simulation

La simulation n'évolue pas en temps réel continu mais de manière discrète, rythmée par des cycles appelés "ticks". La méthode centrale tick() orchestre l'ensemble des événements à chaque unité de temps selon la séquence stricte suivante :

1. Traitement prioritaire des agents sur les nœuds : pour chaque nœud du graphe, le moteur met à jour en priorité absolue les agents ayant le statut « prioritaire », suivis des agents sans priorité.

2. Traitement des agents sur les arêtes : les agents actuellement en cours de transit sur les routes voient ensuite leur déplacement mis à jour.

3. Gestion de la finalisation des trajets : en fin de tick, les agents ayant atteint leur destination sont soit retirés du graphe, soit réassignés à une nouvelle destination aléatoire pour simuler un trafic continu.

C. Mécanique de déplacement

Le déplacement des agents est géré par la méthode **moveAgent**, qui est appelée à chaque tick de la simulation pour chaque agent actif. Le principe repose sur un système de vitesse résiduelle : à chaque tick, l'agent dispose d'un budget de déplacement égal à sa vitesse, auquel s'ajoute la progression déjà accumulée sur l'arête courante. Tant que ce budget n'est pas épuisé, l'agent continue d'avancer. À chaque itération, l'agent calcule le prochain nœud à atteindre en interrogeant son algorithme de **pathfinding**. Chaque agent possède son propre **pathfinder**, avec un fallback sur celui du moteur de simulation si aucun n'est défini.

Avant de s'engager sur une arête, l'agent vérifie sa capacité : si celle-ci est saturée, l'agent passe en état **WAITING** et attend le prochain tick sans perdre sa progression. Si l'arête est disponible, l'agent y est ajouté et la distance est déduite de son budget. Deux cas se présentent alors :

- Si le budget restant couvre toute la distance de l'arête, l'agent atteint le nœud suivant et continue la boucle
- Sinon, la progression partielle est sauvegardée et sera réutilisée au tick suivant

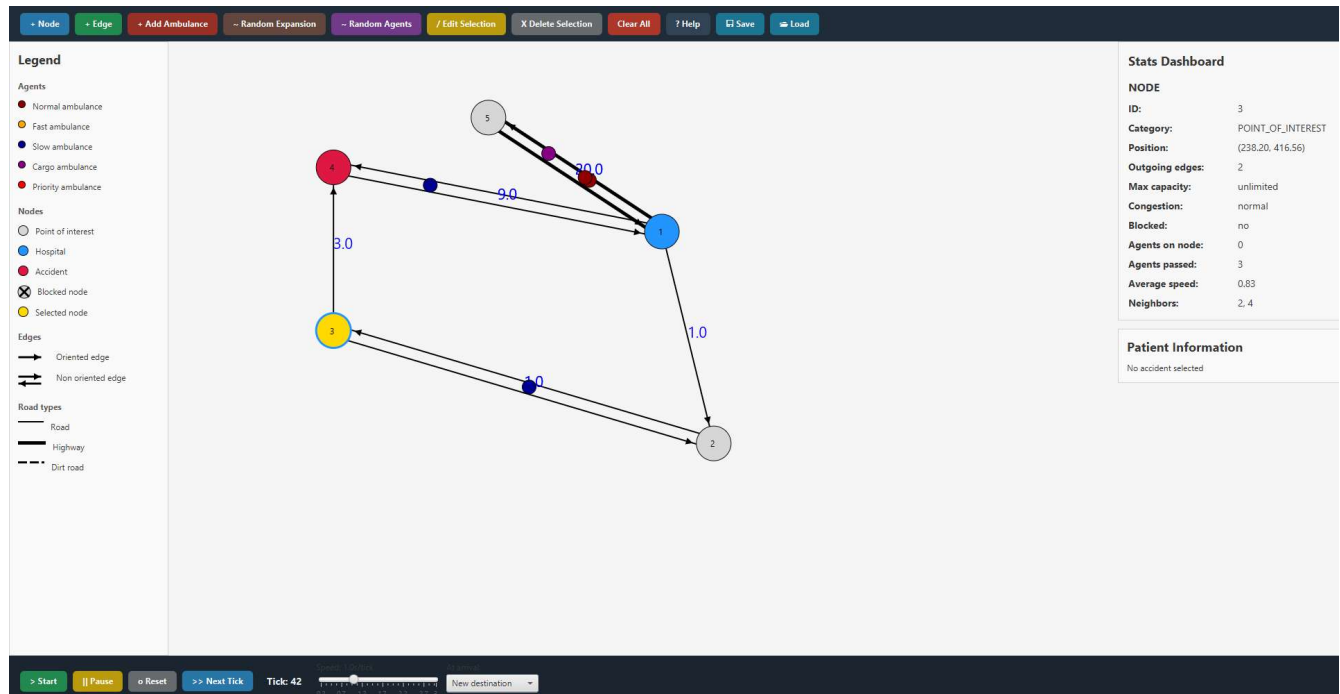
À la fin, l'agent passe en état **ARRIVED** s'il a atteint sa destination, ou reste **MOVING** sinon

Pour garantir l'intégrité de cette mécanique lors de scénarios complexes, des règles de gestion ont été implémentées :

- **Priorisation des flux** : lorsque la capacité d'un nœud d'intersection ou d'une arête est libérée, le moteur trie les agents en attente selon leur type. Les véhicules d'urgence (statut **PRIORITY_AMBULANCE**) obtiennent le droit de passage en priorité absolue, évitant ainsi qu'une ambulance lente (**SLOW_AMBULANCE**) ou standard ne bloque une intervention critique.
- **Gestion de la congestion** : lorsqu'un agent est contraint de passer au statut **WAITING** à cause d'une route saturée, l'arête concernée met à jour son état de congestion. Les algorithmes de recherche de chemin avancés (comme **CongestionAwarePathFinder**) intègrent dynamiquement cette donnée pour recalculer, au tick suivant, un itinéraire alternatif contournant le point de blocage.
- **Comportement à l'arrivée (ArrivalBehavior)** : une fois le statut **ARRIVED** validé au nœud de destination, l'agent déclenche un comportement spécifique configurable. Il peut soit être retiré du graphe (fin de sa mission), soit se voir attribuer immédiatement une nouvelle destination aléatoire parmi les nœuds disponibles, assurant ainsi l'alimentation continue du trafic de la simulation.

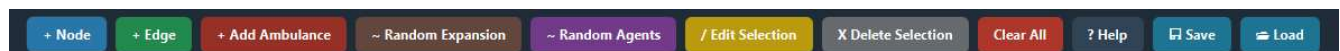
D. Interaction utilisateur

L'interface graphique JavaFX a été conçue pour séparer clairement entre la visualisation du réseau, le contrôle de la simulation et la manipulation des données. Elle se structure en quatre zones principales.



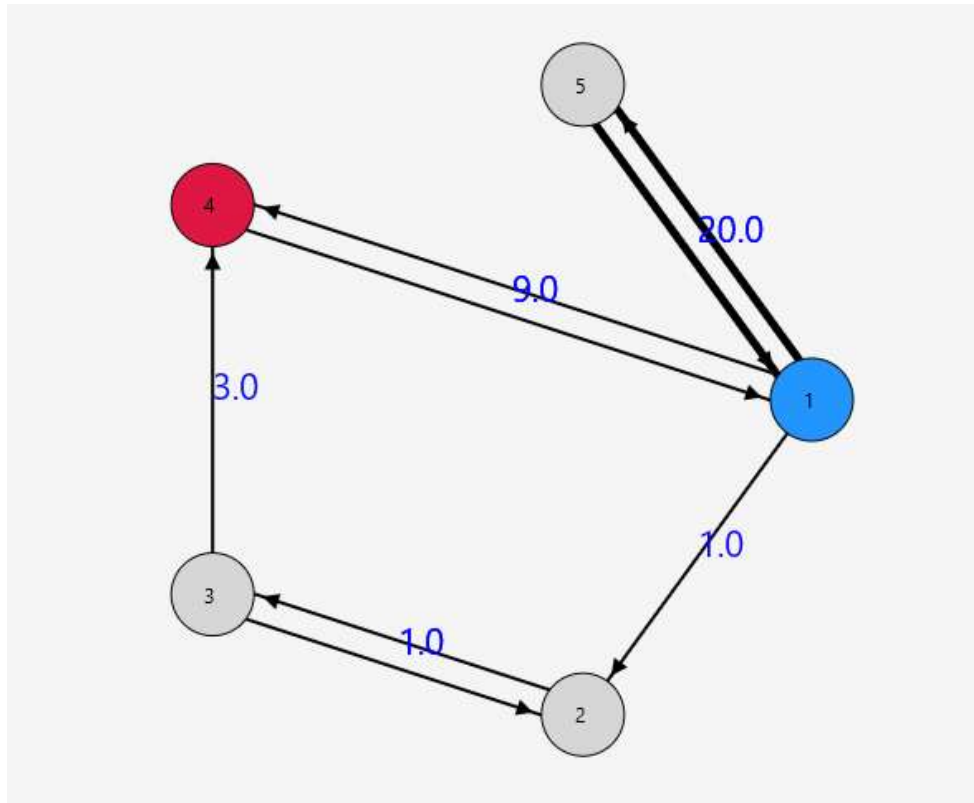
Interface complète *LifeLine GPS*

- **La barre d'outils supérieure :** elle regroupe l'ensemble des fonctionnalités d'édition du réseau et de scénarisation. L'utilisateur peut y activer les modes de création, ajouter une ambulance spécifique, générer des flux de véhicules, modifier ou supprimer des éléments sélectionnés, vider le graphe ou encore sauvegarder et charger un fichier.



Barre d'outils supérieure

- **La zone centrale de visualisation** : c'est le cœur de l'interface. Elle affiche dynamiquement le graphe sous forme de nœuds (cercles colorés numérotés) et d'arêtes (flèches pleines ou pointillées indiquant le sens et le type de route).



Zone centrale de visualisation

Les agents y sont représentés par de petits cercles de couleur se déplaçant en temps réel le long des arêtes. L'utilisateur peut directement interagir avec cette zone à la souris, notamment pour sélectionner un élément ou repositionner un nœud par glisser-déposer.

- Le panneau latéral de contrôle et d'informations : situé à droite, il est divisé en deux sections contextuelles :

Stats Dashboard : affiche en temps réel les propriétés de l'élément sélectionné (ID, catégorie, coordonnées, arêtes sortantes, capacité maximale, etc.).

Patient : s'active lorsqu'un nœud de type **ACCIDENT** est sélectionné. Il affiche les données médicales et contextuelles du patient généré (nom, âge, rythme cardiaque, etc.).

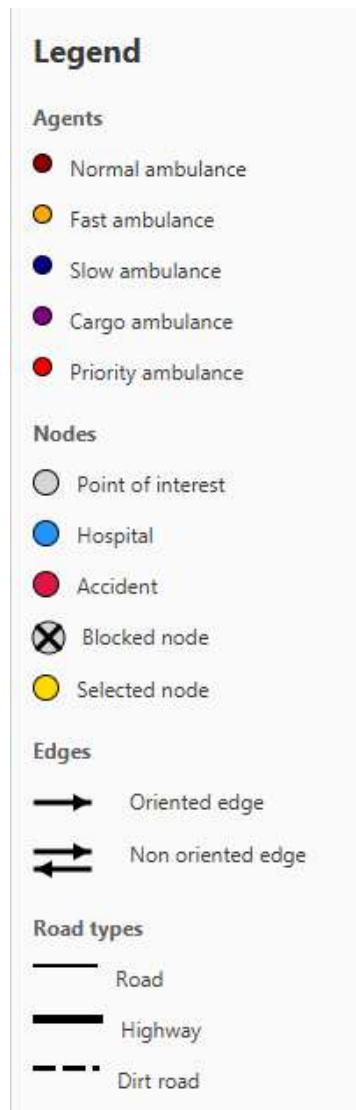
Stats Dashboard	
NODE	
ID:	4
Name:	Serious personal injury
Category:	ACCIDENT
Position:	(238.20, 181.44)
Outgoing edges:	1
Max capacity:	unlimited
Congestion:	normal
Blocked:	no
Agents on node:	0
Agents passed:	4
Average speed:	0.75
Neighbors:	1

Section statistiques

Patient	
Accident type:	Traffic Accident
Description:	Car crash near the intersection.
Name:	Jean Serane
Age:	42
BPM:	116
Temperature:	37.7 °C
Conscious:	yes
Condition:	Injured but conscious

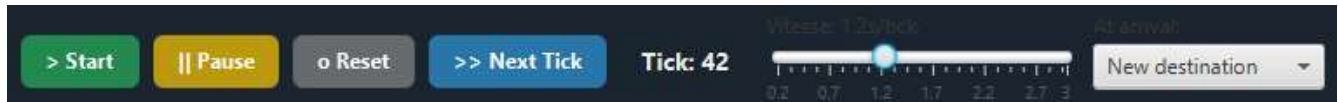
Section patient

- Un panneau **Legend** situé à gauche indique ce que représentent les formes de l'interface (types d'ambulances, états des nœuds, etc.).



Panneau de légende

- **La barre de simulation inférieure** : elle permet de piloter l'exécution temporelle du moteur de simulation. Elle intègre les commandes standard de lecture ainsi qu'un bouton de pas-à-pas pour analyser le comportement des agents tick par tick. Un indicateur textuel affiche le tick actuel, tandis qu'un slider de vitesse permet de régler la cadence de rafraîchissement de la simulation. Enfin, un menu déroulant permet de configurer le comportement global des agents à leur arrivée à destination.



Barre de simulation inférieure

IV. Difficultés rencontrées

Pendant la réalisation de notre projet, nous avons rencontré quelques difficultés, dont voici les principales :

Perte de vitesse à l'arrivée d'un nœud en cours de chemin

Lors de la traversée d'un chemin multi-arêtes, un agent dont la vitesse lui permettait de franchir plusieurs arêtes en un seul tick s'arrêtait systématiquement à chaque nœud intermédiaire. Le surplus de vitesse non utilisé était simplement perdu. La solution a consisté à restructurer la méthode `moveAgent()` autour d'une boucle `while (remainingSpeed > 0)` qui réutilise le surplus de vitesse pour progresser immédiatement sur l'arête suivante dans le même tick.

Indépendance des deux arêtes orientées

Le graphe étant non orienté, chaque arête $A \rightarrow B$ génère automatiquement une arête inverse $B \rightarrow A$ comme objet Java distinct. Modifier le poids de $A \rightarrow B$ via l'interface ne modifiait pas $B \rightarrow A$, ce qui provoquait des comportements incohérents selon le sens de déplacement de l'agent. La correction a consisté à propager systématiquement toute modification de propriété d'une arête à son arête inverse correspondante.

ComboBox d'algorithmes qui écrase le comportement des agents

Initialement, le pathfinder était global à l'engine et partagé par tous les agents. Lorsqu'on changeait l'algorithme dans l'interface, cela affectait rétroactivement tous les agents en cours de déplacement. La solution a été d'attribuer à chaque agent son propre `IPathFinder` au moment de sa création, l'engine conservant un pathfinder global uniquement comme fallback pour les agents qui n'en ont pas.

Sélection des agents impossible en état WAITING

Pendant la simulation, les agents en état WAITING ne pouvaient pas être sélectionnés visuellement. Le problème venait de `updateAgent()` qui réécrivait systématiquement l'état de l'agent à MOVING après chaque appel à `moveAgent()`, écrasant ainsi l'état WAITING positionné en interne. La correction a consisté à conditionner cette réaffectation : l'état n'est mis à jour que si l'agent n'est pas déjà en WAITING.

V. Pistes d'améliorations

Maintenant, nous présentons différentes pistes d'amélioration pour enrichir notre application :

Prise en charge des patients

La principale piste d'amélioration du projet est la gestion du cycle de prise en charge d'un patient. Lorsqu'une ambulance intervient sur un nœud accident, le patient reste associé à ce nœud même après l'arrivée des secours. Une évolution intéressante serait de permettre à l'ambulance de récupérer le patient et de le transporter jusqu'à un hôpital. Une fois l'accident résolu, le nœud accident pourrait automatiquement redevenir un point d'intérêt classique.

Visualisation des chemins

Une amélioration intéressante indiquée dans le cahier des charges aurait été d'afficher visuellement le chemin calculé pour chaque ambulance avec une flèche du nœud de départ au nœud d'arrivée. Nous ne l'avons pas implémentée par manque de temps et nous avons préféré garder un code fonctionnel plutôt que de prendre le risque de le casser.

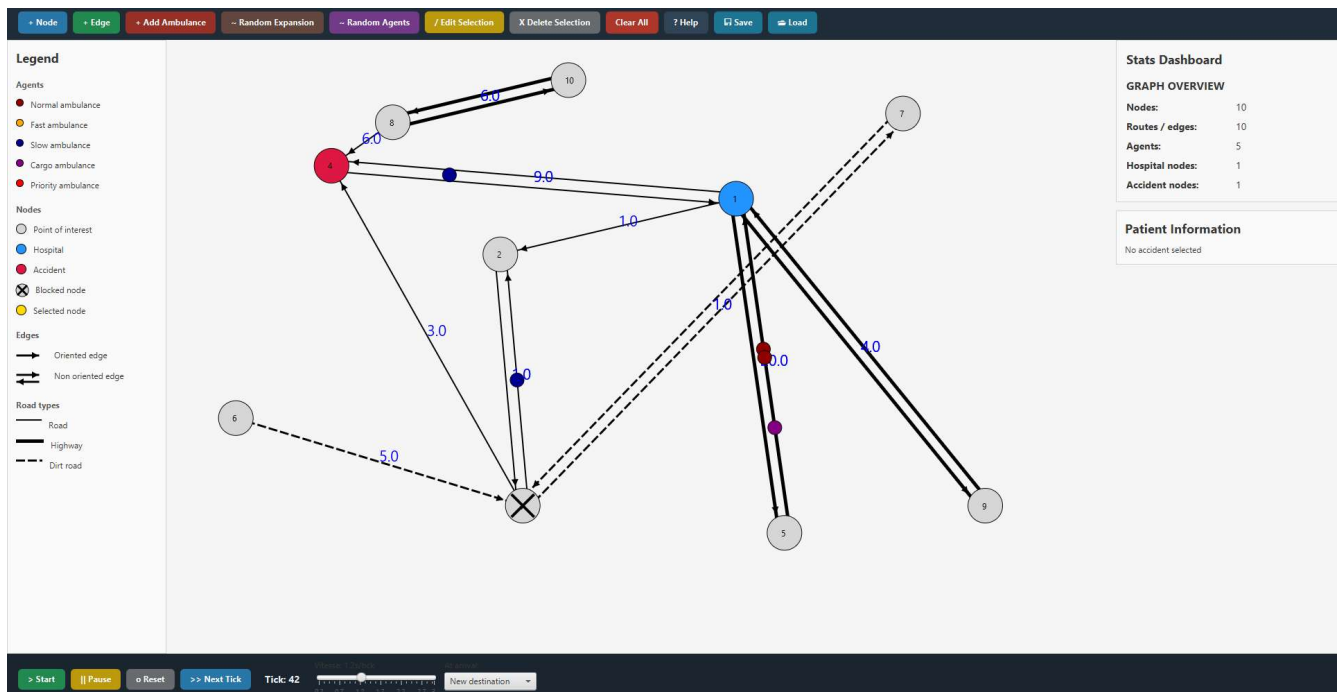
Gradient de couleur

Nous avons envisagé d'utiliser un dégradé de couleur pour représenter la congestion. Nous avons préféré conserver un affichage plus simple et plus stable afin de nous concentrer sur les fonctionnalités principales du projet.

Icône d'application

L'ajout d'icônes png JavaFX s'est révélé plus compliqué que prévu à cause de la gestion des ressources et des différences entre Eclipse, le terminal et les fichiers JAR. Malgré plusieurs recherches sur Stack Overflow et avec l'IA, les solutions trouvées n'étaient pas toujours compatibles avec notre configuration.

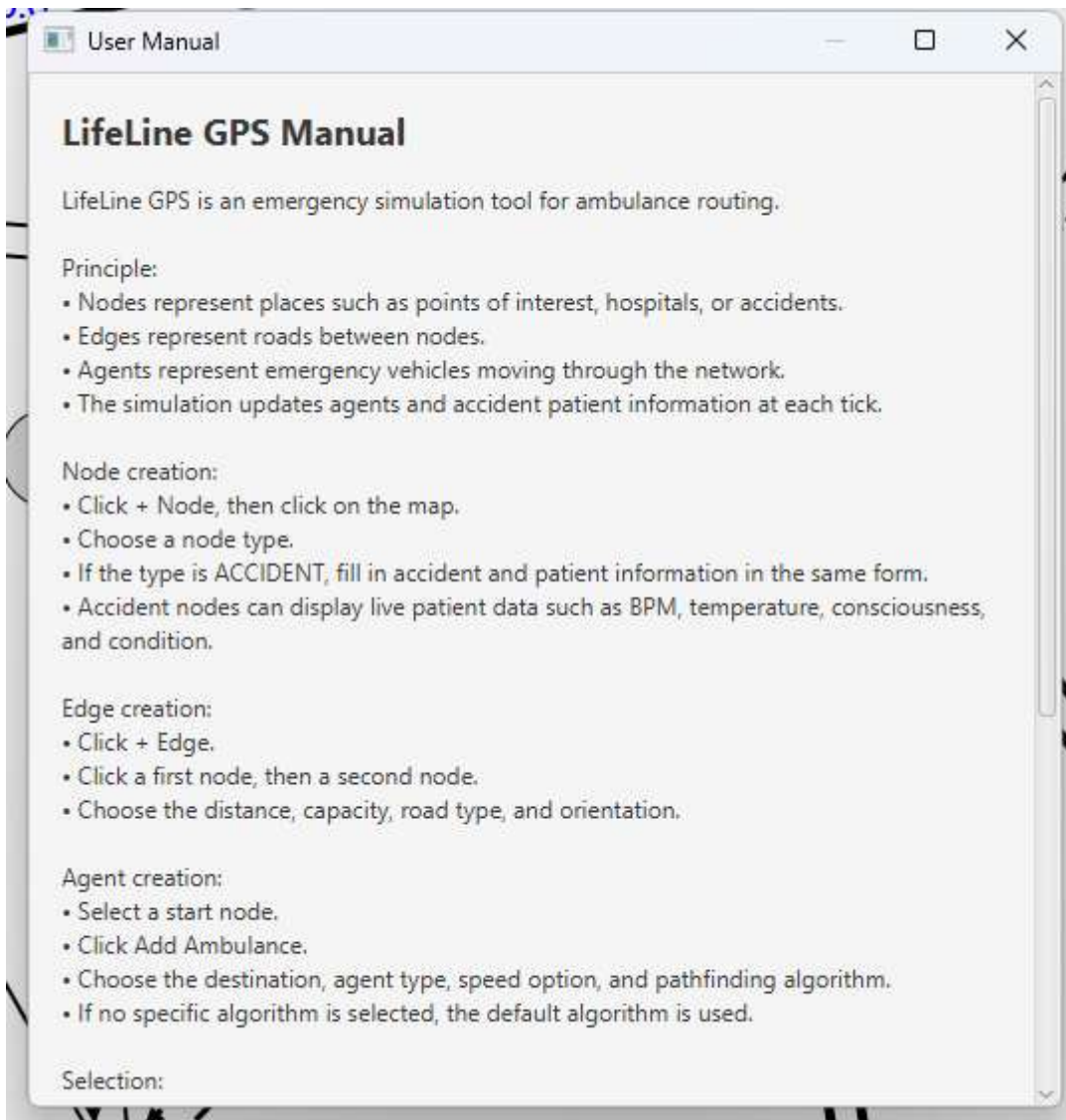
VI. Résultat final



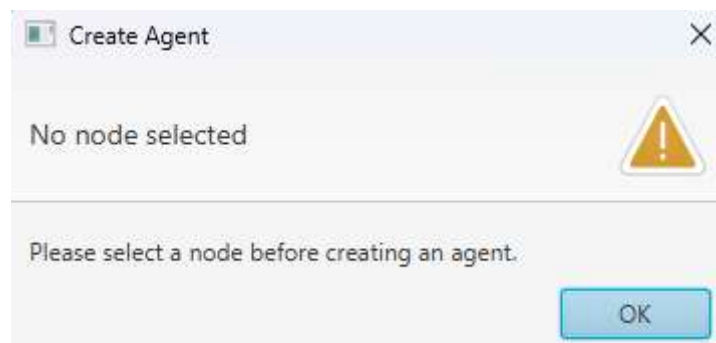
Interface finale *LifeLine GPS*

The 'Create Node' dialog box shows the 'Node properties' section. The 'Name' field contains 'Optional name'. The 'Node type' is set to 'ACCIDENT'. Under 'Accident information', the 'Accident type' is 'Traffic Accident'. The 'Description' field contains 'Accident description'. The 'Patient name' field contains 'Patient name'. The 'Age' field contains '0', 'BPM' contains '80', and 'Temperature' contains '37.0'. The 'State' is checked as 'Conscious' and the 'Condition' is 'Stable'. 'OK' and 'Cancel' buttons are at the bottom.

Menu de création d'un nœud



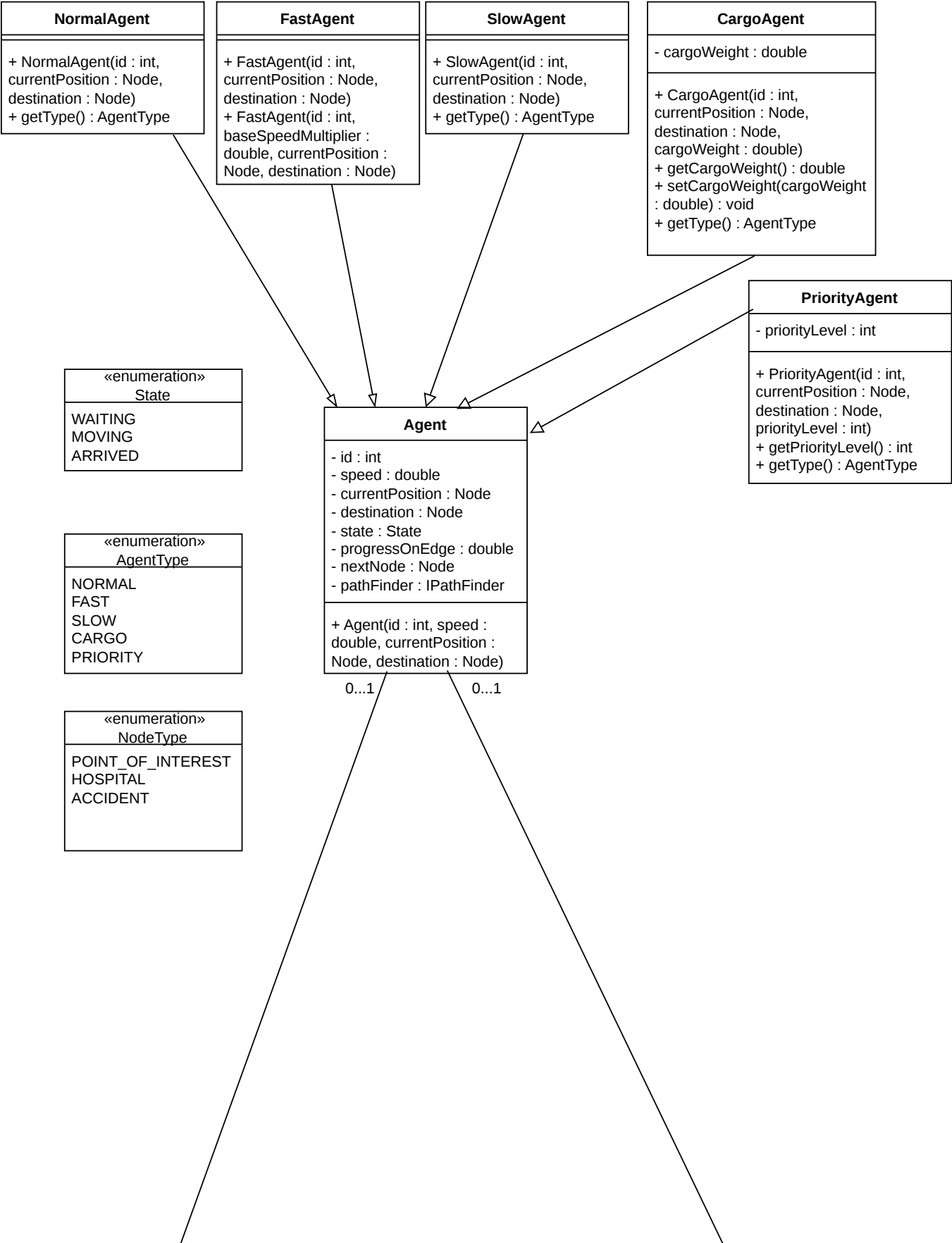
Manuel utilisateur

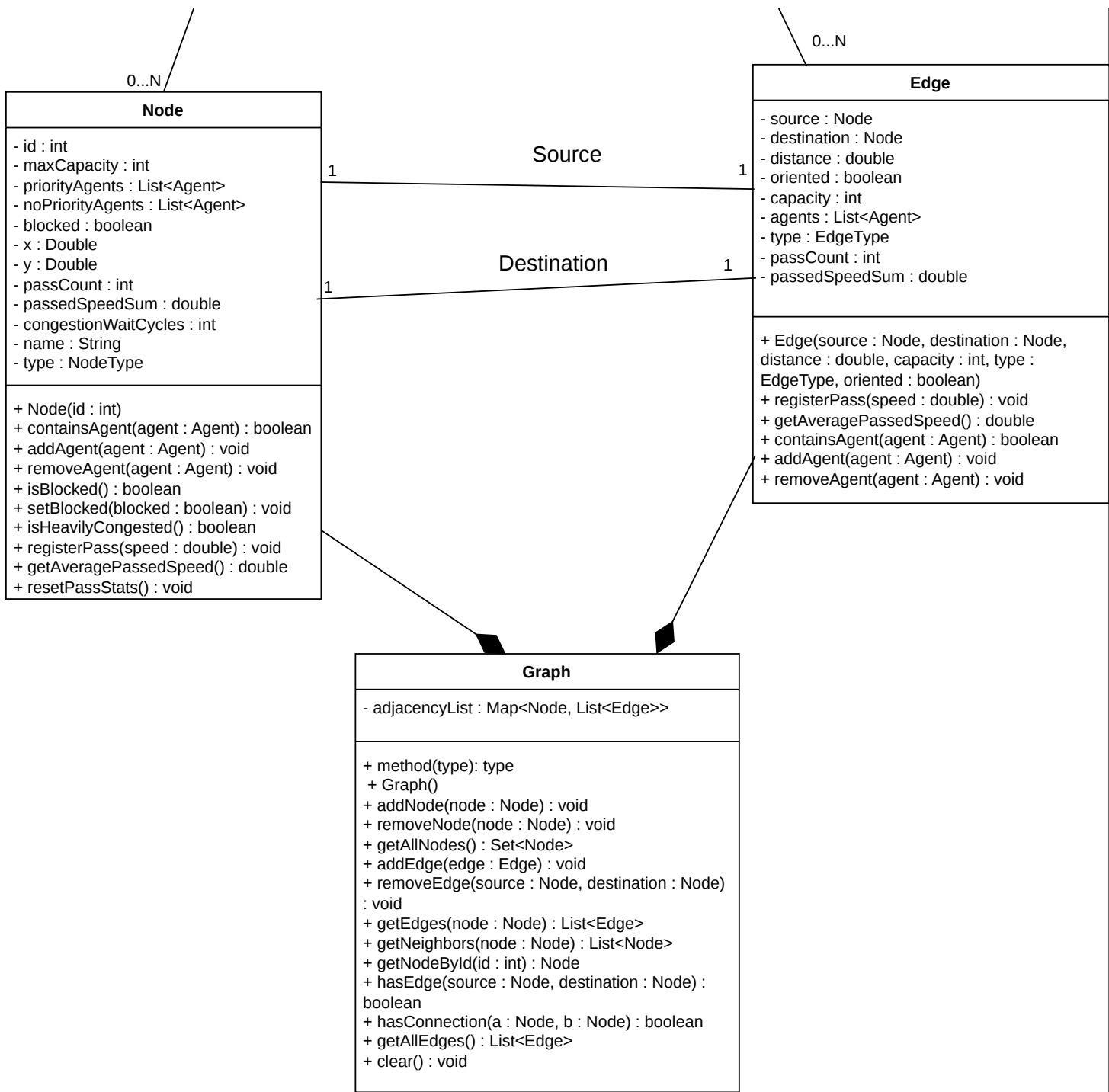


Indication d'une erreur à l'utilisateur

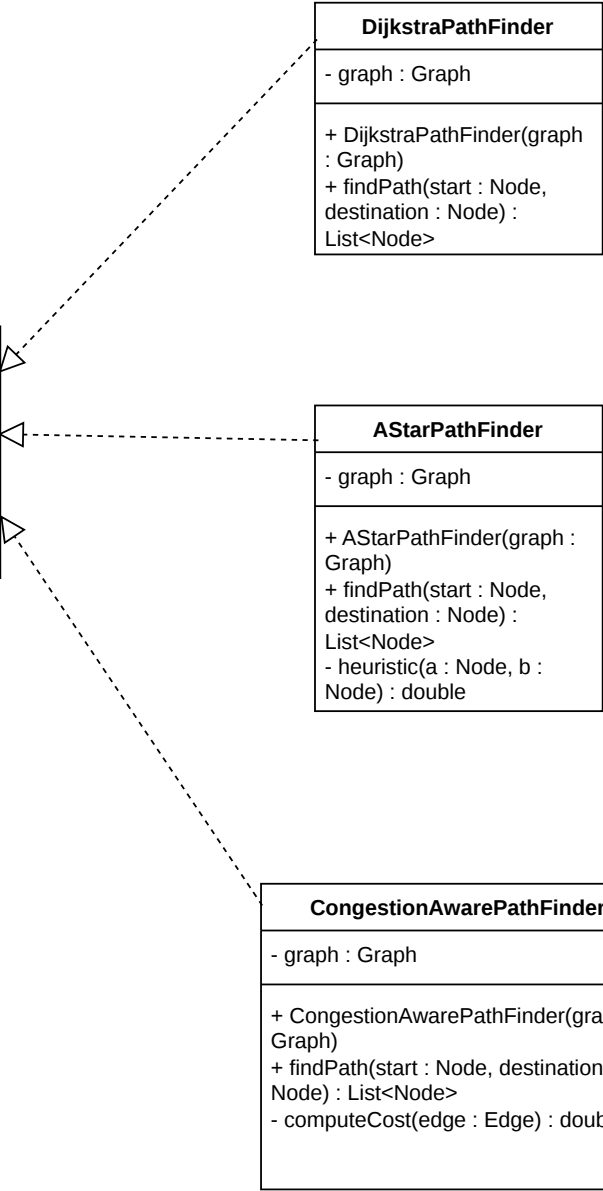
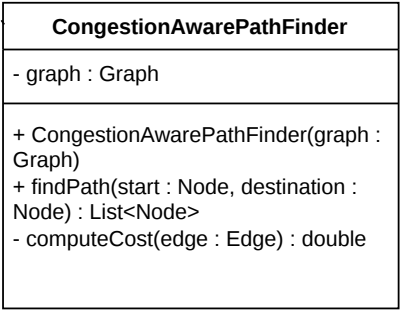
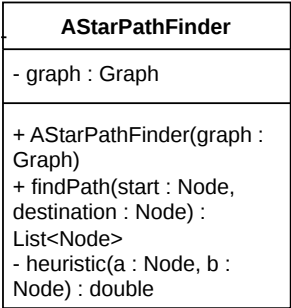
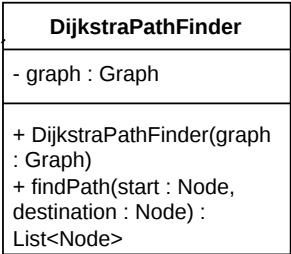
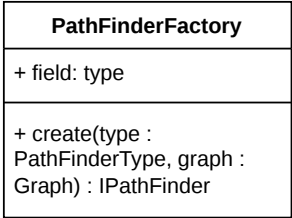
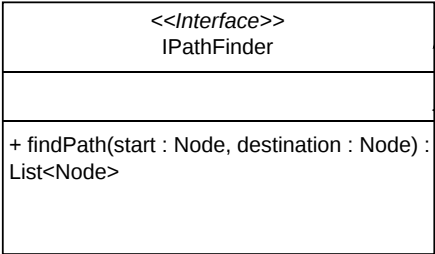
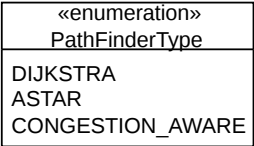
Pour conclure, *LifeLine GPS* constitue une base solide et fonctionnelle pour simuler des comportements de trafic. Le cœur de la simulation (moteur tick-based, pathfinding avec Dijkstra, A*, congestion aware, et gestion des capacités et des types de routes) est opérationnel et testé. L'interface permet une interaction complète et dynamique avec le graphe. Des fonctionnalités restent cependant à intégrer : propriétés attractives ou répulsives des nœuds, visualisation de la densité par gradient de couleurs. Ces extensions constitueraient une évolution naturelle du projet vers un vrai outil de simulation de trafic d'urgence. Ce projet nous a avant tout appris à travailler en équipe sur une base de code commune, à gérer les dépendances entre modules, et à prendre des décisions d'architecture sous la contrainte du temps, des compétences aussi importantes que le code lui-même.

Model





Pathfinding



Simulation

«enumeration» ArrivalBehavior
RANDOM_DESTINATION REMOVE

SimulationEngine
- graph : Graph - agents : List<Agent> - currentTick : long - arrivalBehavior : ArrivalBehavior - pathFinder : IPathFinder
+ SimulationEngine(graph : Graph, pathFinder : IPathFinder) + tick() : void + moveAgent(agent : Agent) : void + addAgent(agent : Agent) : void + removeAgent(agent : Agent) : void + reset() : void + clearAgents() : void

